

Clase 5_2:

Programación

Programación Orientada a Objetos

Temari

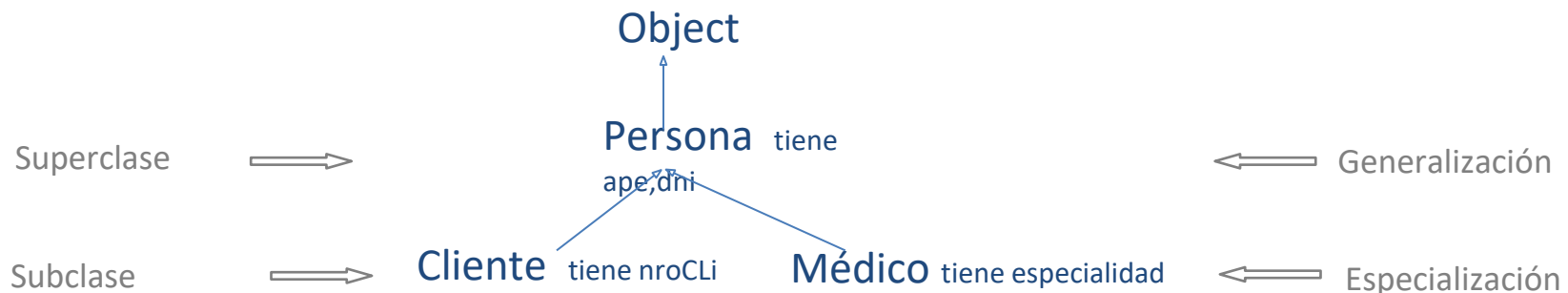
-
-
- • Herencia continuación • • • • •
- Sobreescritura.
- • • • • • • • • •
- Polimorfismo.
- • • • • • • • • •
-

Repaso de Herencia.

La herencia nos permite **concebir una nueva clase (subclase)** de objetos **como refinamiento** de **otra clase** de objetos ya existente (superclase), conservando las similitudes entre las clases de objetos y especificando solamente las diferencias de la nueva clase de objetos. Se arma un árbol jerárquico de clases.

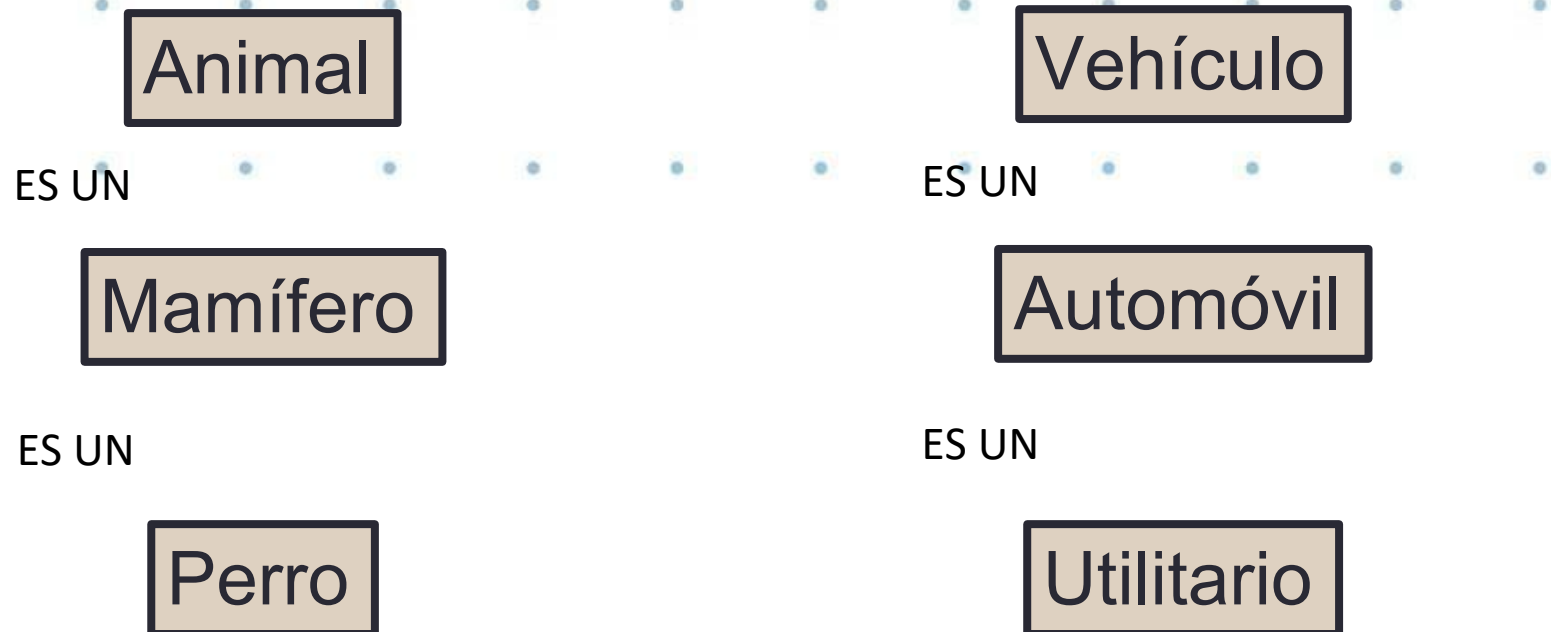
Las **subclases** pueden **agregar nuevos atributos** y comportamientos y además pueden **cambiar el comportamiento de algunos métodos heredados**.

Jerarquía de clases



Repaso Herencia

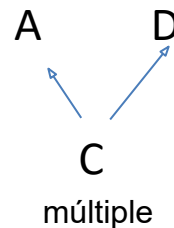
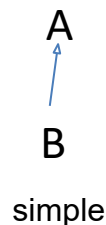
La herencia implementa la relación “*es un*” entre clases: nos permite establecer que una clase es como otra, con la excepción de que la nueva clase incluye estado y comportamiento extra.



Repaso Herencia.

La herencia puede ser:

- **De estado: se heredan los atributos** de la superclase.
- **De comportamiento: se heredan los métodos** de la superclase.
- **Total o parcial: se heredan todos los atributos y métodos(total), o solo parte de ellos.**
- **Simple o múltiple: se hereda de una sola superclase, o se hereda de varias superclases.**



Repaso Herencia.

Cuando se programa usando herencia, se **programa por extensión:**

- **Se extiende el código existente**, se reutiliza código.
- No se vuelve a escribir lo heredado. Sólo se agrega lo correspondiente a los nuevos atributos y a las operaciones nuevas.
- Cuando **se le envía un mensaje a un objeto, el S.O. lo busca en la clase de dicho objeto. Si no lo encuentra**, porque es heredado, **sube por el árbol jerárquico** de clases aplicando una BÚSQUEDA LOOK-UP, hasta encontrarlo y lo ejecuta.

Herencia Múltiple

- Python provee la posibilidad de definir relaciones de herencia múltiple o sea heredar de más de una clase.

```
class Terrestre:  
    def desplazar(self):  
        print ("El animal anda")
```

```
class Acuatico:  
    def desplazar(self):  
        print ("El animal nada")
```

```
class Cocodrilo(Terrestre, Acuatico):  
    def __init__(self, lugar):  
        Terrestre.__init__(self)  
        Acuatico.__init__(self)  
        self.__origen=lugar
```

Herencia Múltiple

¿Qué método se ejecuta?

```
c = Cocodrilo("Florida")  
c.desplazar()
```



El animal anda

Toma el método de la clase más a la izquierda
de la lista de parámetros, por eso muestra:
“El animal anda”

Herencia Múltiple

La herencia múltiple **conlleva un problema**, y es que **si varias super clases tienen los mismos atributos o métodos**, la subclase sólo podrá heredar de una de ellas lo que está duplicado.

En estos casos **Python dará prioridad a las clases más a la izquierda** en el momento de la declaración de la subclase (ver el ejemplo previo con el método desplazar).

Herencia Múltiple

El constructor de la subclase deberá invocar a los constructores de sus superclases:

```
class Cocodrilo (Terrestre,  
    Acuatico):  
    def __init__(self,  
        lugar):  
        Terrestre.__init__(self)  
        Acuatico.__init__(self)  
        self. origen=lugar
```

Así cada constructor inicializa las variables de instancia que hereda de la clase correspondiente y luego el constructor de la subclase solo inicializa las propias.

Sobreescritura

- ¿Qué pasa si en una subclase se define un método que tenga exactamente la misma firma (nombre, tipo de retorno, y lista de argumentos (orden, cantidad y tipo)) que un método de la superclase?
 - La subclase está sobreescribiendo el método de la superclase.
- La sobreescritura permite definir un método en una subclase, con la misma firma que un método de la superclase.

Ejemplo de Sobreescritura

```
class A:
```

```
    def __init
```

```
        ____(self): self.
```

```
        __x = 1
```

```
    def m1(self):
```

```
        print("m1 de A")
```

A

↑

B

```
class B(A):
```

```
    def __init__(self):
```

```
        super().__
```

```
        __init__()
```

```
        self.__y = 1
```

```
    def m1(self):
```

```
        print("m1 de B")
```

```
c = B()    #crea un objeto de la clase B
```

```
c.m1()    # imprime "m1 de B"
```

Ejemplo de Sobreescritura

Lo que se logra definiendo el método `m1()` en la Clase Derivada (B) se conoce como *Método Overriding*.

Cuando se cree el objeto "c" y se llame al método `m1()`, este será tomado de la propia clase B y no de la superclase.

Si comentamos el método `m1()` de la clase B (Clase Derivada) y corremos nuevamente el código, el método llamado será el `m1()` de la Clase A.

Polimorfismo

- El polimorfismo es la **capacidad** que poseen **objetos de diferentes clases** para **responder al mismo mensaje de diferentes maneras**.

En general esto **ocurre** en una **jerarquía** de **clases** donde las **subclases** **sobreescriben** **métodos** heredados.

Polimorfismo

- Para utilizar la **técnica del polimorfismo** generalmente se realiza lo siguiente:
- **Se define una jerarquía** de herencia.
 - **Se implementa una interfaz común** en la superclase que se encuentra en el **tope** de la **jerarquía**.
 - **Se codifican** en las subclases **diferentes versiones de los métodos** especificados en la **interfaz común**. (**overriding**)

Polimorfismo - ejemplo

```
import seaborn as sb
```

```
class Grafico:  
    def dibujar(self, datos, color):  
        pass
```

```
class DispersionCategorica(Grafico):  
    def dibujar(self, datos):  
        sb.catplot(datos, x="cut", y="carat", color="green")
```

```
class Curva(Grafico):  
    def dibujar(self, datos):  
        sb.lineplot(datos, x="cut", y="carat", color="orange")
```

```
class Pares(Grafico):  
    def dibujar(self, datos):  
        sb.pairplot(datos, vars = ["cut", "carat"])
```

Polimorfismo - ejemplo

```
datos = sb.load_dataset("diamonds")
```

```
graficos = [Curva(), Pares(), DispersionCategorica()]
```

```
for grafico in graficos:  
    grafico.dibujar(datos)
```

Polimorfismo

```
class Grafico:
    def dibujar(self, datos, color):
        pass

class DispersionCategorica(Grafico):
    def dibujar(self, datos):
        sb.catplot(datos, x="cut", y="carat", color="green")

class Curva(Grafico):
    def dibujar(self, datos):
        sb.lineplot(datos, x="cut", y="carat", color="orange")

class Pares(Grafico):
    def dibujar(self, datos):
        sb.pairplot(datos, vars = ["cut", "carat"])
```

```
datos = sb.load_dataset("diamonds")

graficos = [Curva(), Pares(), DispersionCategorica()]
for grafico in graficos:
    grafico.dibujar(datos)
```

Polimorfismo

En el ejemplo, tenemos **tres subclases** distintas: de Dispersion Categórica, Curva y Pares que **heredan una superclase**

Llamada Gráfico

La clase **Grafico** define el método **dibujar()** sin implementación, el cual **es sobrescrito en cada una de las subclases** de Grafico.

El **envío del mensaje dibujar()** se traducirá en **distintos comportamientos según la instancia que reciba.**

El **polimorfismo se vale de la sobrescritura de métodos y binding dinámico** para poder funcionar.

Ejercicio 1

partiendo de este programa

```
class Perro ( Mascota)
    def __init__(self, nom, ed, due,
        raza): super().__init__(nom,ed, due)
        self._raza=raza
```

```
    def verRaza(self):
        return self._raza
    def modRaza(self, otra):
        self._raza=otra
```

```
class Gato (Mascota):
    def __init__(self, nom, ed,
        due,colorpelo): super().__init__( nom, ed, due)
        self._pelaje=colorpelo
    def verPelo(self):
        return self._pelaje
    def modPelo(self,otr
    oP): self._pelaje=otr
        oP
```

Ejercicio 1

¿Hay algún método que pueda ser polimórfico?

Ejercicio 1

```
class Perro (Mascota):  
    def hablarConElDuenio(self):  
        print ('yo ladro')
```

```
class Gato (Mascota):  
    def hablarConElDuenio(self):  
        print ('yo mauullo')
```

Ejercicio 2

Haga un programa que instancie diferentes mascotas, las guarde en una lista y luego recorra la colección haciendo "hablar" a las mascotas

Ejercicio 2

Haga un programa que instancie diferentes mascotas, las guarde en una lista y luego recorra la colección haciendo "hablar" a las mascotas

Ejercicio 2

```
mascotas = []
```

```
co= Gato("michifuz", 3, 'pedro',  
'blanco') mascotas.append(co)
```

```
ga=Gato ("Firulai", 5, 'pedro',  
'atigrado') mascotas.append(ga)
```

```
pe=Perro ("Fido", 8, 'pablo',  
"Dálmata") mascotas.append(pe)
```

```
pe1=Perro('Tobi',10, 'luis', 'Dogo')  
mascotas.append(pe1)
```

Polimorfismo

```
for m in mascotas:
```

```
    m.hablarConElDuenio()
```

"yo mauullo"

"yo mauullo"

"yo ladro"

"yo ladro"

Ejemplo 3

Figuras cerradas en el plano, con un método para calcular el área.

La clase base (Figura()) no tiene comportamiento definido ni atributos, dado

Que:

- Cada figura tiene atributos muy distintos (radio en el caso del círculo, base y altura en el caso del triángulo, etc.),
- Para el cálculo del área, cada figura tiene una fórmula diferente

```
class Figura(object):  
    """ Una figura en el plano. """  
    def area(self):  
        " Este método debe ser redefinido. "  
        pass
```

Ejemplo 3

```
from math import pi
class Circulo(Figura):
    """ Un círculo en el plano. """
    def __init__(self, radio=0):
        " Constructor de círculo. "
        self.radio = radio

    def area(self):
        " Devuelve el área del círculo. "
        return pi * self.radio * self.radio
```

Binding dinámico

Ejemplo 3

```
class Triangulo(Figura):  
    """ Un triángulo en el plano. """  
    def __init__(self, base=0, altura=0):  
        " Constructor de triángulo. "  
        self.base = base  
        self.altura = altura  
  
    def area(self):  
        " Devuelve el área del triángulo. "  
        return self.base * self.altura / 2.
```

Ejemplo 3

Se ejecutan las siguientes sentencias:

```
t = Triangulo(3, 5)  
t.area()
```

```
c = Circulo(4)  
c.area()
```

Resultados:

✓
0 s



```
t = Triangulo(3, 5)  
t.area()
```

7.5

✓
0 s

```
[114] c = Circulo(4)  
c.area()
```

50.26548245743669

